

Memoria de Prácticas: Algorítmica Básica (P4)

1. Diseño del Algoritmo

El problema consiste en formar equipos de 3 personas a partir de N participantes (donde N es un múltiplo de 3) minimizando el conflicto total.

Representación

Se utiliza una lista de participantes libres para el seguimiento del estado del sistema durante la exploración.

Árbol de búsqueda

En cada nivel del árbol se fija el primer participante libre (p_1) y se generan todas las parejas posibles (p_2, p_3) a partir del resto de los integrantes para formar un equipo válido.

Funciones

- **Coste:** Suma de los conflictos bidireccionales del equipo seleccionado, calculada mediante la fórmula:
$$C_{ij} + C_{ji} + C_{ik} + C_{ki} + C_{jk} + C_{kj}$$
- **Poda:** Se utiliza un límite inferior basado en la media de la matriz de conflictos para descartar ramas no óptimas de forma temprana:
$$\text{conflicto_poda} = \text{conflicto_equipo} + \text{media} * \text{len}(\text{participantes_restantes})$$

2. Implementación

El código está estructurado en módulos independientes para facilitar la lectura de ficheros, la lógica de backtracking con poda y la resolución mediante programación lineal.

Tipos de datos

Se utilizan listas nativas para gestionar los participantes y una combinación de diccionarios junto con listas de listas para la representación de la matriz de conflictos.

Estructura

El programa principal (**formarEquipos.py**) utiliza una función recursiva llamada `generar_todas_las_soluciones`, la cual aplica la poda descrita antes de realizar la llamada recursiva para reducir drásticamente el espacio de estados.

3. Experimentación

Se han realizado diversas pruebas con diferentes tamaños de N para evaluar el rendimiento general del algoritmo.

Tabla de Resultados (Backtracking)

Problema (N)	Tiempo (ms)	Nodos	Valor Óptimo
3	0.016	2	27
6	0.036	15	45
9	0.202	100	44
12	53.190	32,396	148

Análisis: El crecimiento del tiempo de computación es exponencial respecto a N . Sin embargo, la estrategia de poda propuesta demuestra ser extremadamente efectiva, reduciendo el número de nodos explorados significativamente en comparación con una búsqueda exhaustiva tradicional (evitando aproximadamente el 99% de los nodos en las instancias de mayor tamaño).

4. Enfoque con Programación Lineal

Para contrastar los resultados del algoritmo de búsqueda, se formalizó el problema como un modelo de **Programación Lineal Entera (PLE)**:

- **Formalización:** Se definen variables binarias x_{ijk} que toman el valor 1 si se llega a formar el equipo compuesto por la terna $\{i, j, k\}$, y 0 en caso contrario.
- **Restricción:** La suma de x_{ijk} para todas las ternas donde participe el miembro p debe ser igual a 1 ($\sum x_{ijk} = 1$, para cada participante p), garantizando que cada persona pertenezca a un único equipo.

Comparativa: La implementación se realizó utilizando la API de **PuLP**. Si bien el enfoque de programación lineal es matemáticamente robusto, para estas instancias de pequeño tamaño el algoritmo de backtracking con poda resultó ser **6.71 veces más rápido** (BT: 119ms frente a PL: 800ms). Esto se debe principalmente al coste fijo de inicialización y carga del solver externo.

5. Conclusiones

- El algoritmo de ramificación y poda diseñado se muestra altamente eficiente para valores de $N \leq 9$.
- El uso de la media de la matriz de conflictos como cota inferior representa una heurística simple, pero sumamente potente para la poda del árbol.
- La Programación Lineal se establece como una alternativa robusta y excelente para validar la corrección de los resultados, a pesar de que su overhead de inicialización penalice el rendimiento en problemas pequeños.